adlio/schema

Embedded schema migration package for Go

ref: v1.4.0

License: MIT License

Stars: 42

Forks: 5

This PDF was generated by gitprint.me

Top Contributors

adlio (163)

dependabot[bot] (13)

adlio-amzn (9)

kalafut (2)

Chapter 0.0.0

root

README.md

Schema - Database Migrations for Go

An embeddable library for applying changes to your Go application's database/sql schema.

Features

- Cloud-friendly design tolerates embedded use in clusters
- Supports migrations in embed.FS (requires go:embed in Go 1.16+)

- Depends only on Go standard library (Note that all go.mod dependencies are used only in tests)
- Unidirectional migrations (no “down” migration complexity)

Usage Instructions

Create a schema.Migrator in your bootstrap/config/database connection code, then call its Apply() method with your database connection and a slice of *schema.Migration structs.

The .Apply() function figures out which of the supplied Migrations have not yet been executed in the database (based on the ID), and executes the Script for each in **alphabetical order by IDe**.

The []*schema.Migration can be created manually, but the package has some utility functions to make it easier to parse .sql files into structs with the filename as the ID and the file contents as the Script.

Using go:embed (requires Go 1.16+)

Go 1.16 added features to embed a directory of files into the binary as an embedded filesystem (embed.FS).

Assuming you have a directory of SQL files called my-migrations/ next to your main.go file, you’ll run something like this:

```
//go:embed my-migrations
var MyMigrations embed.FS

func main() {
   db, err := sql.Open(...) // Or however you get a *sql.DB

   migrations, err := schema.FSMigrations(MyMigrations, "my-migrations/*.sql")
   migrator := schema.NewMigrator(schema.WithDialect(schema.MySQL))
   err = migrator.Apply(db, migrations)
}
```

The WithDialect() option accepts: schema.MySQL, schema.Postgres, schema.SQLite or schema.MSSQL. These dialects all use only database/sql calls, so you may have success with other databases which are SQL-compatible with the above dialects.

You can also provide your own custom Dialect. See dialect.go for the definition of the Dialect interface, and the optional Locker interface. Note that Locker is critical for clustered operation to ensure that only one of many processes is attempting to run migrations simultaneously.

Using Inline Migration Structs

If you’re running in an earlier version of Go, Migration{} structs will need to be created manually:

```
db, err := sql.Open(...)

migrator := schema.NewMigrator() // Postgres is the default Dialect
migrator.Apply(db, []*schema.Migration{
   &schema.Migration{
      ID: "2019-09-24 Create Albums",
      Script: `
      CREATE TABLE albums (
         id SERIAL PRIMARY KEY,
         title CHARACTER VARYING (255) NOT NULL
      )
      `
   },
})
```

Constructor Options

The NewMigrator() function accepts option arguments to customize the dialect and the name of the migration tracking table. By default, the tracking table will be named schema_migrations. To change it to my_migrations instead:

```
migrator := schema.NewMigrator(schema.WithTableName("my_migrations"))
```

It is theoretically possible to create multiple Migrators and to use mutliple migration tracking tables within the same application and database.

It is also OK for multiple processes to run Apply on identically configured migrators simultaneously. The Migrator only creates the tracking table if it does not exist, and then locks it to modifications while building and running the migration plan. This means that the first-arriving process will **win** and will perform its migrations on the database.

Supported Databases

This package was extracted from a PostgreSQL project. Other databases have solid automated test coverage, but should be considered somewhat experimental in production use cases. Contributions are welcome for additional databases or feature enhancements / bug fixes.

- [x] PostgreSQL (database/sql driver only, see adlio/pgxschema if you use jack/pgx)
- [x] SQLite (thanks kalafut!)
- [x] MySQL / MariaDB
- [x] SQL Server
- [ ] CockroachDB, Redshift, Snowflake, etc (open a Pull Request)

Package Opinions

There are many other schema migration tools. This one exists because of a particular set of opinions:

- Database credentials are runtime configuration details, but database schema is a **build-time applicaton dependency**, which means it should be “compiled in” to the build, and should not rely on external tools.
- Using an external command-line tool for schema migrations needlessly complicates testing and deployment.
- SQL is the best language to use to specify changes to SQL schemas.
- “Down” migrations add needless complication, aren’t often used, and are tedious to properly test when they are used. In the unlikely event you need to migrate backwards, it’s possible to write the “rollback” migration as a separate “up” migration.
- Deep dependency chains should be avoided, especially in a compiled binary. We don’t want to import an ORM into our binaries just to get SQL the features of this package. The schema package imports only standard library packages (**NOTE** *We do import ory/dockertest in our tests).
- Sequentially-numbered integer migration IDs will create too many unnecessary schema collisions on a distributed, asynchronously-communicating team (this is not yet strictly enforced, but may be later).

Rules of Applying Migrations

-

Never, ever change the ID (filename) or Script (file contents) of a Migration which has already been executed on your database. If you’ve made a mistake, you’ll need to correct it in a subsequent migration.

-

Use a consistent, but descriptive format for migration IDs/filenames. Consider prefixing them with today’s timestamp. Examples:

```
ID: "2019-01-01T13:45:00 Creates Users"
ID: "2001-12-18 001 Changes the Default Value of User Affiliate ID"
```

Do not use simple sequentialnumbers like ID: "1".

Migration Ordering

Migrations **are not** executed in the order they are specified in the slice. They will be re-sorted alphabetically by their IDs before executing them.

Contributions

… are welcome. Please include tests with your contribution. We've integrated dockertest to automate the process of creating clean test databases.

Before contributing, please read the package opinions section. If your contribution is in disagreement with those opinions, then there's a good chance a different schema migration tool is more appropriate.

Roadmap

- [x] Enhancements and documentation to facilitate asset embedding via go:embed
- [ ] Add a Validate() method to allow checking migration names for consistency and to detect problematic changes in the migrations list.
- [x] SQL Server support
- [x] SQL Server support for the Locker interface to protect against simultaneous migrations from clusters of servers.

Version History

1.4.0 - Feb 21, 2026

- **Breaking:** Go 1.24+ now required (was 1.22)
- **Breaking:** MSSQL driver changed from denisenkom/go-mssqldb to microsoft/go-mssqldb
- Update all dependencies to latest versions
- Fix data race in MSSQL Unlock function
- Migrate CI from CircleCI to GitHub Actions
- Add automated release workflow

1.3.9 - Jul 21, 2025

- Update Go version requirement to 1.21 to resolve CircleCI build issues with slices package dependency
- Update CircleCI configuration to use Go 1.21

1.3.8 - Jul 19, 2025

- Update golang.org/x/crypto to v0.40.0 to address security vulnerabilities
- Update golang.org/x/net to v0.42.0 to address security vulnerabilities

1.3.7 - Jul 19, 2025

- Add SQL Server support for the Locker interface using sp_getapplock/sp_releaseapplock
- Fix SQL Server transaction handling for concurrent migrations

1.3.4 - Apr 9, 2023

- Update downstream dependencies to address vulnerabilities in test dependencies.

1.3.3 - Jun 19, 2022

- Update downstream dependencies of ory/dockertest due to security issues.

1.3.0 - Mar 25, 2022

- Basic SQL Server support (no locking, not recommended for use in clusters)
- Improved support for running tests on ARM64 machines (M1 Macs)

1.2.3 - Dec 10, 2021

- BUGFIX: Restore the ability to chain NewMigrator().Apply

1.2.2 - Dec 9, 2021

- Add support for migrations in an embed.FS (FSMigrations(filesystem fs.FS, glob string))
- Add MySQL/MariaDB support (experimental)
- Add SQLite support (experimental)

- Update go.mod to go 1.17.

1.1.14 - Nov 18, 2021

Security patches in upstream dependencies.

1.1.13 - May 22, 2020

Bugfix for error with advisory lock being held open. Improved test coverage for simultaneous execution.

1.1.11 - May 19, 2020

Use a database-held lock for all migrations not just the initial table creation.

1.1.9 - May 17, 2020

Add the ability to attach a logger.

1.1.8 - Nov 24, 2019

Switch to filepath package for improved cross-platform filesystem support.

1.1.7 - Oct 1, 2019

Began using pg_advisory_lock() to prevent race conditions when multiple processes/machines try to simultaneously create the migrations table.

1.1.1 - Sep 28, 2019

First published version.

LICENSE

MIT License

Copyright (c) 2021 Aaron Longwell

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
}

applied-migration.go

```
package schema

import (
    "fmt"
    "time"
)

// AppliedMigration represents a successfully-executed migration. It embeds
// Migration, and adds fields for execution results. This type is what
// records persisted in the schema_migrations table align with.
type AppliedMigration struct {
    Migration

    // Checksum is the MD5 hash of the Script for this migration
    Checksum string

    // ExecutionTimeInMillis is populated after the migration is run, indicating
    // how much time elapsed while the Script was executing.
    ExecutionTimeInMillis int64

    // AppliedAt is the time at which this particular migration's Script began
    // executing (not when it completed executing).
    AppliedAt time.Time
}

// GetAppliedMigrations retrieves all already-applied migrations in a map keyed
// by the migration IDs
func (m Migrator) GetAppliedMigrations(db Queryer) (applied
map[string]*AppliedMigration, err error) {
    applied = make(map[string]*AppliedMigration)

    // Get the raw data from the Dialect
    migrations, err := m.Dialect.GetAppliedMigrations(m.ctx, db, m.QuotedTableName())
    if err != nil {
        err = fmt.Errorf("Failed to GetAppliedMigrations. Did somebody change the
structure of the %s table? %w", m.QuotedTableName(), err)
        return applied, err
    }

    // Re-index into a map
    for _, migration := range migrations {
        applied[migration.ID] = migration
    }

    return applied, err
}
}
```

applied-migration_test.go

```
package schema

import (
    "testing"

    sqlmock "github.com/DATA-DOG/go-sqlmock"
```

```
)

func TestGetAppliedMigrations(t *testing.T) {
    withEachTestDB(t, func(t *testing.T, tdb *TestDB) {
        db := tdb.Connect(t)
        defer func() { _ = db.Close() }()

        migrator := makeTestMigrator(WithDialect(tdb.Dialect))
        migrations := testMigrations(t, "useless-ansi")
        err := migrator.Apply(db, migrations)
        if err != nil {
            t.Error(err)
        }

        expectedCount := len(migrations)
        applied, err := migrator.GetAppliedMigrations(db)
        if err != nil {
            t.Error(err)
        }
        if len(applied) != expectedCount {
            t.Errorf("Expected %d applied migrations. Got %d", expectedCount,
len(applied))
        }
    })
}

func TestGetAppliedMigrationsErrorsWhenTheTableDoesntExist(t *testing.T) {
    withEachTestDB(t, func(t *testing.T, tdb *TestDB) {
        db := tdb.Connect(t)
        defer func() { _ = db.Close() }()

        migrator := makeTestMigrator()
        migrations, err := migrator.GetAppliedMigrations(db)
        if err == nil {
            t.Error("Expected an error. Got none.")
        }
        if len(migrations) > 0 {
            t.Error("Expected empty list of applied migrations")
        }

    })
}

func TestGetAppliedMigrationsHasFriendlyScanError(t *testing.T) {
    withEachTestDB(t, func(t *testing.T, tdb *TestDB) {
        migrator := makeTestMigrator(WithDialect(tdb.Dialect))

        db, mock, err := sqlmock.New()
        if err != nil {
            t.Error(err)
        }

        // Build a rowset that is completely different than the AppliedMigration
        // struct is expecting to force a Scan error
        rows := sqlmock.NewRows([]string{"nonsense", "column", "names"}).AddRow(1,
"trash", "data")
        mock.ExpectQuery("^SELECT").RowsWillBeClosed().WillReturnRows(rows)

        _, err = migrator.GetAppliedMigrations(db)
        expectErrorContains(t, err, migrator.TableName)
    })
}
}
```

dialect.go

```
package schema

import "context"

// Dialect defines the minimal interface for a database dialect. All dialects
// must implement functions to create the migrations table, get all applied
// migrations, insert a new migration tracking record, and perform escaping
// for the tracking table's name
type Dialect interface {
    QuotedTableName(schemaName, tableName string) string

    CreateMigrationsTable(ctx context.Context, tx Queryer, tableName string) error
    GetAppliedMigrations(ctx context.Context, tx Queryer, tableName string) (applied
[]*AppliedMigration, err error)
    InsertAppliedMigration(ctx context.Context, tx Queryer, tableName string, migration
*AppliedMigration) error
}

// Locker defines an optional Dialect extension for obtaining and releasing
// a global database lock during the running of migrations. This feature is
// supported by PostgreSQL and MySQL, but not SQLite.
type Locker interface {
    Lock(ctx context.Context, tx Queryer, tableName string) error
    Unlock(ctx context.Context, tx Queryer, tableName string) error
}
}
```

doc.go

```
// Package schema provides tools to manage database schema changes
// ("migrations") as embedded functionality inside another application
// which is using a database/sql
//
// Basic usage instructions involve creating a schema.Migrator via the
// schema.NewMigrator() function, and then passing your *sql.DB
// to its .Apply() method.
//
// See the package's README.md file for more usage instructions.
package schema
}
```

embed_go116.go

```
//go:build go1.16
// +build go1.16

package schema

import (
```

```
    "fmt"
    "io/fs"
)

// FSMigrations receives a filesystem (such as an embed.FS) and extracts all
// files matching the provided glob as Migrations, with the filename (without
extension)
// being the ID and the file's contents being the Script.
//
// Example usage:
//
//  FSMigrations(embeddedFS, "my-migrations/*.sql")
func FSMigrations(filesystem fs.FS, glob string) (migrations []*Migration, err error) {
    migrations = make([]*Migration, 0)

    entries, err := fs.Glob(filesystem, glob)
    if err != nil {
        return migrations, fmt.Errorf("failed to process glob '%s' in embed.FS: %w",
glob, err)
    }

    for _, entry := range entries {
        migration := &Migration{
            ID: MigrationIDFromFilename(entry),
        }
        data, err := fs.ReadFile(filesystem, entry)
        if err != nil {
            return migrations, err
        }
        migration.Script = string(data)
        migrations = append(migrations, migration)
    }
    return migrations, nil
}
}
```

embed_go116_test.go

```
//go:build go1.16
// +build go1.16

package schema

import (
    "embed"
    "io/fs"
    "testing"
    "testing/fstest"
)

//go:embed test-migrations
var exampleMigrations embed.FS

func TestMigrationsFromEmbedFS(t *testing.T) {
    migrations, err := FSMigrations(exampleMigrations, "test-migrations/saas/*.sql")
    if err != nil {
        t.Error(err)
    }

    expectedCount := 2
```

```
    if len(migrations) != expectedCount {
        t.Errorf("Expected %d migrations, got %d", expectedCount, len(migrations))
    }

    SortMigrations(migrations)
    expectID(t, migrations[0], "2019-01-01 0900 Create Users")
    expectScriptMatch(t, migrations[0], `^CREATE TABLE users`)
    expectID(t, migrations[1], "2019-01-03 1000 Create Affiliates")
    expectScriptMatch(t, migrations[1], `^CREATE TABLE affiliates`)
}

func TestMigrationsWithInvalidGlob(t *testing.T) {
    _, err := FSMigrations(exampleMigrations, "/a/path[]with/bad/glob/pattern")
    expectErrorContains(t, err, "/a/path[]with/bad/glob/pattern")
}

func TestFSMigrationsWithInvalidFiles(t *testing.T) {
    testfs := fstest.MapFS{
        "invalid-migrations": &fstest.MapFile{
            Mode: fs.ModeDir,
        },
        "invalid-migrations/real.sql": &fstest.MapFile{
            Data: []byte("File contents"),
        },
        "invalid-migrations/fake.sql": nil,
    }
    _, err := FSMigrations(testfs, "invalid-migrations/*.sql")
    expectErrorContains(t, err, "fake.sql")
}
}
```

errors_test.go

```
package schema

import (
    "context"
    "database/sql"
    "errors"
    "fmt"
    "strings"
    "testing"

    sqlmock "github.com/DATA-DOG/go-sqlmock"
)

var (
    // ErrConnFailed indicates that the Conn() method failed (couldn't get a
connection)
    ErrConnFailed = fmt.Errorf("connect failed")

    // ErrBeginFailed indicates that the Begin() method failed (couldn't start Tx)
    ErrBeginFailed = fmt.Errorf("begin failed")

    ErrLockFailed = fmt.Errorf("lock failed")
)

// BadQueryer implements the Connection interface, but fails on every call to
// Exec or Query. The error message will include the SQL statement to help
// verify the "right" failure occurred.
```

```
type BadQueryer struct{}

func (bq BadQueryer) ExecContext(ctx context.Context, sql string, args ...interface{})
(sql.Result, error) {
    return nil, fmt.Errorf("FAIL: %s", strings.TrimSpace(sql))
}

func (bq BadQueryer) QueryContext(ctx context.Context, sql string, args ...interface{})
(*sql.Rows, error) {
    return nil, fmt.Errorf("FAIL: %s", strings.TrimSpace(sql))
}

// BadDB implements the interface for the *sql.DB Conn() method in a way that
// always fails
type BadDB struct{}

func (bd BadDB) Conn(ctx context.Context) (*sql.Conn, error) {
    return nil, ErrConnFailed
}

func TestApplyWithNilDBProvidesHelpfulError(t *testing.T) {
    withEachDialect(t, func(t *testing.T, d Dialect) {
        migrator := NewMigrator(WithDialect(d))
        err := migrator.Apply(nil, testMigrations(t, "useless-ansi"))
        if !errors.Is(err, ErrNilDB) {
            t.Errorf("Expected %v, got %v", ErrNilDB, err)
        }
    })
}

func TestApplyWithNoMigrations(t *testing.T) {
    db, _, _ := sqlmock.New()
    migrator := NewMigrator()
    err := migrator.Apply(db, []*Migration{})
    if err != nil {
        t.Errorf("Expected no error when running no migrations, got %s", err)
    }
}
func TestApplyConnFailure(t *testing.T) {
    bd := BadDB{}
    migrator := Migrator{}
    err := migrator.Apply(bd, testMigrations(t, "useless-ansi"))
    if err != ErrConnFailed {
        t.Errorf("Expected %v, got %v", ErrConnFailed, err)
    }
}

func TestApplyLockFailure(t *testing.T) {
    migrator := NewMigrator()
    db, mock, _ := sqlmock.New()
    mock.ExpectExec("^SELECT pg_advisory_lock").WillReturnError(ErrLockFailed)
    err := migrator.Apply(db, testMigrations(t, "useless-ansi"))
    if err != ErrLockFailed {
        t.Errorf("Expected err '%s', got '%s'", ErrLockFailed, err)
    }
}

func TestApplyBeginFailure(t *testing.T) {
    migrator := NewMigrator()

    db, mock, _ := sqlmock.New()
    mock.ExpectExec("^SELECT pg_advisory_lock").WillReturnResult(sqlmock.NewResult(0,
0))
    mock.ExpectBegin().WillReturnError(ErrBeginFailed)
```

```
    mock.ExpectExec("^SELECT pg_advisory_unlock").WillReturnResult(sqlmock.NewResult(0,
0))
    err := migrator.Apply(db, testMigrations(t, "useless-ansi"))
    if err != ErrBeginFailed {
        t.Errorf("Expected err '%s', got '%s'", ErrBeginFailed, err)
    }
}

func TestApplyCreateFailure(t *testing.T) {
    migrator := NewMigrator()

    db, mock, _ := sqlmock.New()
    mock.ExpectExec("^SELECT pg_advisory_lock").WillReturnResult(sqlmock.NewResult(0,
0))
    mock.ExpectBegin()
    expectedErr := fmt.Errorf("CREATE TABLE statement failed")
    mock.ExpectExec("^CREATE TABLE").WillReturnError(expectedErr)
    mock.ExpectRollback()
    mock.ExpectExec("^SELECT pg_advisory_unlock").WillReturnResult(sqlmock.NewResult(0,
0))
    err := migrator.Apply(db, testMigrations(t, "useless-ansi"))
    if err != expectedErr {
        t.Errorf("Expected err '%s', got '%s'", expectedErr, err)
    }
}

func TestLockFailure(t *testing.T) {
    bq := BadQueryer{}
    migrator := NewMigrator()
    err := migrator.lock(bq)
    expectErrorContains(t, err, "SELECT pg_advisory_lock")
}

func TestUnlockFailure(t *testing.T) {
    bq := BadQueryer{}
    migrator := NewMigrator()
    err := migrator.unlock(bq)
    expectErrorContains(t, err, "SELECT pg_advisory_unlock")
}

func TestComputeMigrationPlanFailure(t *testing.T) {
    bq := BadQueryer{}
    withEachDialect(t, func(t *testing.T, d Dialect) {
        migrator := NewMigrator(WithDialect(d))
        _, err := migrator.computeMigrationPlan(bq, []*Migration{})
        expectErrorContains(t, err, "FAIL: SELECT id, checksum,
execution_time_in_millis, applied_at")
    })
}

func expectErrorContains(t *testing.T, err error, contains string) {
    t.Helper()
    if err == nil {
        t.Errorf("Expected an error string containing '%s', got nil", contains)
    } else if !strings.Contains(err.Error(), contains) {
        t.Errorf("Expected an error string containing '%s', got '%s' instead",
contains, err.Error())
    }
}
}
```

file.go

```
package schema

import (
    "fmt"
    "io"
    "os"
    "path"
    "path/filepath"
    "strings"
)

// MigrationIDFromFilename removes directory paths and extensions
// from the filename to make a friendlier Migration ID
func MigrationIDFromFilename(filename string) string {
    return strings.TrimSuffix(filepath.Base(filename), filepath.Ext(filename))
}

// MigrationsFromDirectoryPath retrieves a slice of Migrations from the
// contents of the directory. Only .sql files are read
func MigrationsFromDirectoryPath(dirPath string) (migrations []*Migration, err error) {
    migrations = make([]*Migration, 0)

    // Assemble a glob of the .sql files in the directory. This can
    // only fail if the dirPath itself contains invalid glob characters
    filenames, err := filepath.Glob(filepath.Join(dirPath, "*.sql"))
    if err != nil {
        return migrations, err
    }

    // Friendly failure: if the user provides a valid-looking, but nonexistent
    // directory, we want to error instead of returning an empty set
    if _, err := os.Stat(dirPath); os.IsNotExist(err) {
        return migrations, fmt.Errorf("migrations directory does not exist: %w", err)
    }

    for _, filename := range filenames {
        migration, err := MigrationFromFilePath(filename)
        if err != nil {
            return migrations, err
        }
        migrations = append(migrations, migration)
    }
    return
}

// MigrationFromFilePath creates a Migration from a path on disk
func MigrationFromFilePath(filename string) (migration *Migration, err error) {
    migration = &Migration{}
    migration.ID = MigrationIDFromFilename(filename)
    contents, err := os.ReadFile(path.Clean(filename))
    if err != nil {
        return migration, fmt.Errorf("failed to read migration from '%s': %w",
filename, err)
    }
    migration.Script = string(contents)
    return migration, err
}

// File wraps the standard library io.Read and os.File.Name methods
type File interface {
    Name() string
```

```
    Read(b []byte) (n int, err error)
}

// MigrationFromFile builds a migration by reading from an open File-like
// object. The migration's ID will be based on the file's name. The file
// will *not* be closed after being read.
func MigrationFromFile(file File) (migration *Migration, err error) {
    migration = &Migration{}
    migration.ID = MigrationIDFromFilename(file.Name())
    content, err := io.ReadAll(file)
    if err != nil {
        return migration, err
    }
    migration.Script = string(content)
    return migration, err
}
}
```

file_test.go

```
package schema

import (
    "errors"
    "os"
    "testing"
)

func TestMigrationFromFilePath(t *testing.T) {
    migration, err := MigrationFromFilePath("./test-migrations/saas/2019-01-01 0900
Create Users.sql")
    if err != nil {
        t.Error(err)
    }
    expectID(t, migration, "2019-01-01 0900 Create Users")
    expectScriptMatch(t, migration, `^CREATE TABLE users`)
}

func TestMigrationFromFilePathWithInvalidPath(t *testing.T) {
    _, err := MigrationFromFilePath("./test-migrations/saas/nonexistent-file.sql")
    if err == nil {
        t.Errorf("Expected failure when reading from nonexistent file")
    }
}

func TestMigrationFromFile(t *testing.T) {
    file, err := os.Open("./test-migrations/saas/2019-01-01 0900 Create Users.sql")
    if err != nil {
        t.Error(err)
    }
    migration, err := MigrationFromFile(file)
    if err != nil {
        t.Error(err)
    }
    expectID(t, migration, "2019-01-01 0900 Create Users")
    expectScriptMatch(t, migration, `^CREATE TABLE users`)
}

func TestMigrationsFromDirectoryPath(t *testing.T) {
    migrations, err := MigrationsFromDirectoryPath("./test-migrations/saas")
```

```
    if err != nil {
        t.Error(err)
    }
    SortMigrations(migrations)
    expectID(t, migrations[0], "2019-01-01 0900 Create Users")
    expectID(t, migrations[1], "2019-01-03 1000 Create Affiliates")
}

func TestMigrationsFromDirectoryPathThrowsErrorForInvalidDirectory(t *testing.T) {
    migrations, err := MigrationsFromDirectoryPath("/a/totally/made/up/directory/path")
    if err == nil {
        t.Error("Expected an error trying to load migrations from a fake directory")
    }
    if len(migrations) > 0 {
        t.Errorf("Expected an empty list of migrations. Got %d", len(migrations))
    }
}

func TestMigrationsFromDirectoryPathThrowsErrorForInvalidGlob(t *testing.T) {
    _, err := MigrationsFromDirectoryPath("/a/path[]with/bad/glob/pattern")
    if err == nil {
        t.Error("Expected an error trying to load migrations from a fake directory")
    }
}

func TestMigrationsFromDirectoryPathThrowsErrorWithUnreadableFiles(t *testing.T) {
    err := os.Chmod("./test-migrations/unreadable/unreadable.sql", 0200)
    if err != nil {
        t.Error(err)
    }
    _, err = MigrationsFromDirectoryPath("./test-migrations/unreadable")
    if err == nil {
        t.Error("Expected a failure when trying to read unreadable file")
    }
    _ = os.Chmod("./test-migrations/unreadable/unreadable.sql", 0644) // #nosec
}

type failedReader int

func (fr failedReader) Name() string {
    return "fakeFile.sql"
}

func (fr failedReader) Read(p []byte) (n int, err error) {
    return 0, errors.New("this reader always fails")
}

func TestMigrationFromFileWithUnreadableFile(t *testing.T) {
    var fr failedReader
    _, err := MigrationFromFile(fr)
    if err == nil {
        t.Error("Expected MigrationFromFile to fail when given erroneous file")
    }
}
}
```

main_test.go

```
package schema
```

```go
import (
    "database/sql"
    "fmt"
    "log"
    "os"
    "runtime"
    "sync"
    "testing"

    "github.com/go-sql-driver/mysql"
    dockertest "github.com/ory/dockertest/v3"
)

// TestMain replaces the normal test runner for this package. It connects to
// Docker running on the local machine and launches testing database
// containers to which we then connect and store the connection in a package
// global variable
func TestMain(m *testing.M) {

    log.Printf("Running tests on GOARCH=%s", runtime.GOARCH)

    pool, err := dockertest.NewPool("")
    if err != nil {
        log.Fatalf("Can't run schema tests. Docker is not running: %s", err)
    }

    // Disable logging for MySQL while we await startup of the Docker containero
    // This avoids "[mysql] unexpected EOF" logging input during the delay
    // while the docker containers launch
    _ = mysql.SetLogger(nullMySQLLogger{})

    var wg sync.WaitGroup
    for name := range TestDBs {
        testDB := TestDBs[name]
        wg.Add(1)
        go func() {
            if testDB.IsRunnable() {
                testDB.Init(pool)
            }
            wg.Done()
        }()
    }
    wg.Wait()

    // Restore the default MySQL logger after we successfully connect
    // So that MySQL Driver errors appear as expected
    _ = mysql.SetLogger(log.New(os.Stderr, "[mysql] ",
log.Ldate|log.Ltime|log.Lshortfile))

    code := m.Run()

    // Purge all the containers we created
    // You can't defer this because os.Exit doesn't execute defers
    for _, info := range TestDBs {
        info.Cleanup(pool)
    }

    os.Exit(code)
}

func withEachDialect(t *testing.T, f func(t *testing.T, d Dialect)) {
    dialects := []Dialect{Postgres, MySQL, SQLite, MSSQL}
    for _, dialect := range dialects {
        t.Run(fmt.Sprintf("%T", dialect), func(t *testing.T) {
```

```
            f(t, dialect)
        })
    }
}

func withEachTestDB(t *testing.T, f func(t *testing.T, tdb *TestDB)) {
    for dbName, tdb := range TestDBs {
        t.Run(dbName, func(t *testing.T) {
            if tdb.IsRunnable() {
                f(t, tdb)
            } else {
                t.Skipf("Not runnable on %s", runtime.GOARCH)
            }
        })
    }
}

func withTestDB(t *testing.T, name string, f func(t *testing.T, tdb *TestDB)) {
    tdb, exists := TestDBs[name]
    if !exists {
        t.Fatalf("Database '%s' doesn't exist. Add it to TestDBs", name)
    }
    f(t, tdb)
}

func connectDB(t *testing.T, name string) *sql.DB {
    info, exists := TestDBs[name]
    if !exists {
        t.Fatalf("Database '%s' doesn't exist.", name)
    }
    db, err := sql.Open(info.Driver, info.DSN())
    if err != nil {
        t.Fatalf("Failed to connect to %s: %s", name, err)
    }
    return db
}
}
```

migration.go

```
package schema

import (
    "crypto/md5" // #nosec MD5 only being used to fingerprint script contents, not for
encryption
    "fmt"
    "sort"
)

// Migration is a yet-to-be-run change to the schema. This is the type which
// is provided to Migrator.Apply to request a schema change.
type Migration struct {
    ID     string
    Script string
}

// MD5 computes the MD5 hash of the Script for this migration so that it
// can be uniquely identified later.
func (m *Migration) MD5() string {
    return fmt.Sprintf("%x", md5.Sum([]byte(m.Script))) // #nosec not being used
```

```
cryptographically
}

// SortMigrations sorts a slice of migrations by their IDs
func SortMigrations(migrations []*Migration) {
    // Adjust execution order so that we apply by ID
    sort.Slice(migrations, func(i, j int) bool {
        return migrations[i].ID < migrations[j].ID
    })
}
}
```

migration_test.go

```
package schema

import (
    "regexp"
    "testing"
)

func TestMD5(t *testing.T) {
    testMigration := Migration{Script: "test"}
    expected := "098f6bcd4621d373cade4e832627b4f6"
    if testMigration.MD5() != expected {
        t.Errorf("Expected '%s', got '%s'", expected, testMigration.MD5())
    }
}

func TestSortMigrations(t *testing.T) {
    migrations := []*Migration{
        {ID: "2020-01-01"},
        {ID: "2021-01-01"},
        {ID: "2000-01-01"},
    }
    expectedOrder := []string{"2000-01-01", "2020-01-01", "2021-01-01"}
    SortMigrations(migrations)
    for i, migration := range migrations {
        if migration.ID != expectedOrder[i] {
            t.Errorf("Expected migration #%d to be %s, got %s", i, expectedOrder[i],
migration.ID)
        }
    }
}

func unorderedMigrations() []*Migration {
    return []*Migration{
        {
            ID: "2021-01-01 002",
            Script: `CREATE TABLE data_table (
                id INTEGER PRIMARY KEY,
                name VARCHAR(255),
                created_at TIMESTAMP
            )`,
        },
        {
            ID:     "2021-01-01 001",
            Script: "CREATE TABLE first_table (first_name VARCHAR(255), last_name
VARCHAR(255))",
        },
```

```
        {
            ID:     "2021-01-01 003",
            Script: `INSERT INTO first_table (first_name, last_name) VALUES ('John',
'Doe')`,
        },
    }
}

func expectID(t *testing.T, migration *Migration, expectedID string) {
    t.Helper()
    if migration.ID != expectedID {
        t.Errorf("Expected Migration to have ID '%s', got '%s' instead", expectedID,
migration.ID)
    }
}

func expectScriptMatch(t *testing.T, migration *Migration, regexpString string) {
    t.Helper()
    re, err := regexp.Compile(regexpString)
    if err != nil {
        t.Fatalf("Invalid regexp: '%s': %s", regexpString, err)
    }
    if !re.MatchString(migration.Script) {
        t.Errorf("Expected migration Script to match '%s', but it did not. Script
was:\n%s", regexpString, migration.Script)
    }
}
}
```

migrator.go

```
package schema

import (
    "context"
    "fmt"
    "time"
)

// Migrator is an instance customized to perform migrations on a particular
// database against a particular tracking table and with a particular dialect
// defined.
type Migrator struct {
    SchemaName string
    TableName  string
    Dialect    Dialect
    Logger     Logger

    ctx context.Context
}

// NewMigrator creates a new Migrator with the supplied
// options
func NewMigrator(options ...Option) *Migrator {
    m := Migrator{
        TableName: DefaultTableName,
        Dialect:   Postgres,
        ctx:       context.Background(),
    }
    for _, opt := range options {
```

```go
        m = opt(m)
    }
    return &m
}

// QuotedTableName returns the dialect-quoted fully-qualified name for the
// migrations tracking table
func (m *Migrator) QuotedTableName() string {
    return m.Dialect.QuotedTableName(m.SchemaName, m.TableName)
}

// Apply takes a slice of Migrations and applies any which have not yet
// been applied against the provided database. Apply can be re-called
// sequentially with the same Migrations and different databases, but it is
// not threadsafe... if concurrent applies are desired, multiple Migrators
// should be used.
func (m *Migrator) Apply(db DB, migrations []*Migration) (err error) {
    // Reset state to begin the migration
    if db == nil {
        return ErrNilDB
    }

    if len(migrations) == 0 {
        return nil
    }

    if m.ctx == nil {
        m.ctx = context.Background()
    }

    // Obtain a concrete connection to the database which will be closed
    // at the conclusion of Apply()
    conn, err := db.Conn(m.ctx)
    if err != nil {
        return err
    }
    defer func() { err = coalesceErrs(err, conn.Close()) }()

    // If the database supports locking, obtain a lock around this migrator's
    // table name with a deferred unlock. Go's defers run LIFO, so this deferred
    // unlock will happen before the deferred conn.Close()
    err = m.lock(conn)
    if err != nil {
        return err
    }
    defer func() { err = coalesceErrs(err, m.unlock(conn)) }()

    tx, err := conn.BeginTx(m.ctx, nil)
    if err != nil {
        return err
    }

    err = m.Dialect.CreateMigrationsTable(m.ctx, tx, m.QuotedTableName())
    if err != nil {
        _ = tx.Rollback()
        return err
    }

    err = m.run(tx, migrations)
    if err != nil {
        _ = tx.Rollback()
        return err
    }
```

```
    err = tx.Commit()

    return err
}

func (m *Migrator) lock(tx Queryer) error {
    if l, isLocker := m.Dialect.(Locker); isLocker {
        err := l.Lock(m.ctx, tx, m.QuotedTableName())
        if err != nil {
            return err
        }
        m.log(fmt.Sprintf("Locked %s at %s", m.QuotedTableName(),
time.Now().Format(time.RFC3339Nano)))
    }
    return nil
}

func (m *Migrator) unlock(tx Queryer) error {
    if l, isLocker := m.Dialect.(Locker); isLocker {
        err := l.Unlock(m.ctx, tx, m.QuotedTableName())
        if err != nil {
            return err
        }
        m.log(fmt.Sprintf("Unlocked %s at %s", m.QuotedTableName(),
time.Now().Format(time.RFC3339Nano)))
    }
    return nil
}

func (m *Migrator) computeMigrationPlan(tx Queryer, toRun []*Migration) (plan
[]*Migration, err error) {
    applied, err := m.GetAppliedMigrations(tx)
    if err != nil {
        return plan, err
    }

    plan = make([]*Migration, 0)
    for _, migration := range toRun {
        if _, exists := applied[migration.ID]; !exists {
            plan = append(plan, migration)
        }
    }

    SortMigrations(plan)
    return plan, err
}

func (m *Migrator) run(tx Queryer, migrations []*Migration) error {
    if tx == nil {
        return ErrNilDB
    }

    plan, err := m.computeMigrationPlan(tx, migrations)
    if err != nil {
        return err
    }

    for _, migration := range plan {
        err = m.runMigration(tx, migration)
        if err != nil {
            return err
        }
    }
```

```
    return nil
}

func (m *Migrator) runMigration(tx Queryer, migration *Migration) error {
    startedAt := time.Now()
    _, err := tx.ExecContext(m.ctx, migration.Script)
    if err != nil {
        return fmt.Errorf("Migration '%s' Failed:\n%w", migration.ID, err)
    }

    executionTime := time.Since(startedAt)
    m.log(fmt.Sprintf("Migration '%s' applied in %s\n", migration.ID, executionTime))

    ms := executionTime.Milliseconds()
    if ms == 0 && executionTime.Microseconds() > 0 {
        // Avoid rounding down to 0 for very, very fast migrations
        ms = 1
    }

    applied := AppliedMigration{}
    applied.ID = migration.ID
    applied.Script = migration.Script
    applied.ExecutionTimeInMillis = ms
    applied.AppliedAt = startedAt
    return m.Dialect.InsertAppliedMigration(m.ctx, tx, m.QuotedTableName(), &applied)
}

func (m *Migrator) log(msgs ...interface{}) {
    if m.Logger != nil {
        m.Logger.Print(msgs...)
    }
}

func coalesceErrs(errs ...error) error {
    for _, err := range errs {
        if err != nil {
            return err
        }
    }
    return nil
}
}
```

migrator_test.go

```
package schema

import (
    "fmt"
    "math/rand"
    "sync"
    "testing"
    "time"
)

// TestCreateMigrationsTable ensures that each dialect and test database can
// successfully create the schema_migrations table.
func TestCreateMigrationsTable(t *testing.T) {
    withEachTestDB(t, func(t *testing.T, tdb *TestDB) {
```

```
        db := tdb.Connect(t)
        defer func() { _ = db.Close() }()

        migrator := makeTestMigrator(WithDialect(tdb.Dialect))
        err := tdb.Dialect.CreateMigrationsTable(migrator.ctx, db,
migrator.QuotedTableName())
        if err != nil {
            t.Errorf("Error occurred when creating migrations table: %s", err)
        }

        // Test that we can re-run it again with no error
        err = tdb.Dialect.CreateMigrationsTable(migrator.ctx, db,
migrator.QuotedTableName())
        if err != nil {
            t.Errorf("Calling createMigrationsTable a second time failed: %s", err)
        }
    })
}

// TestLockAndUnlock tests the Lock and Unlock mechanisms of each dialect and
// test database in isolation from any migrations actually being run.
func TestLockAndUnlock(t *testing.T) {
    withEachTestDB(t, func(t *testing.T, tdb *TestDB) {

        db := tdb.Connect(t)
        defer func() { _ = db.Close() }()

        migrator := makeTestMigrator(WithDialect(tdb.Dialect))

        if _, isLocker := tdb.Dialect.(Locker); isLocker {
            err := migrator.lock(db)
            if err != nil {
                t.Fatal(err)
            }

            err = migrator.unlock(db)
            if err != nil {
                t.Fatal(err)
            }
        }
    })
}

// TestApplyInLexicalOrder ensures that each dialect runs migrations in their
// lexical order rather than the order they were provided in the slice. This is
// also the primary test to assert that the data in the tracking table is
// all correct.
func TestApplyInLexicalOrder(t *testing.T) {
    withEachTestDB(t, func(t *testing.T, tdb *TestDB) {

        db := tdb.Connect(t)
        defer func() { _ = db.Close() }()

        start := time.Now().Truncate(time.Second) // MySQL has only second accuracy, so
we need start/end to span 1 second

        tableName := "lexical_order_migrations"
        migrator := NewMigrator(WithDialect(tdb.Dialect), WithTableName(tableName))
        err := migrator.Apply(db, unorderedMigrations())
        if err != nil {
            t.Error(err)
        }

        end := time.Now().Add(time.Second).Truncate(time.Second) // MySQL has only
```

```
second accuracy, so we need start/end to span 1 second

        applied, err := migrator.GetAppliedMigrations(db)
        if err != nil {
            t.Error(err)
        }
        if len(applied) != 3 {
            t.Errorf("Expected exactly 2 applied migrations. Got %d", len(applied))
        }

        firstMigration := applied["2021-01-01 001"]
        if firstMigration == nil {
            t.Fatal("Missing first migration")
        }
        if firstMigration.Checksum == "" {
            t.Error("Expected non-blank Checksum value after successful migration")
        }
        if firstMigration.ExecutionTimeInMillis < 1 {
            t.Errorf("Expected ExecutionTimeInMillis of %s to be tracked. Got %d",
firstMigration.ID, firstMigration.ExecutionTimeInMillis)
        }
        // Put value in consistent timezone to aid error message readability
        appliedAt := firstMigration.AppliedAt.Round(time.Second)
        if appliedAt.IsZero() || appliedAt.Before(start) || appliedAt.After(end) {
            t.Errorf("Expected AppliedAt between %s and %s, got %s", start, end,
appliedAt)
        }
        assertZonesMatch(t, start, appliedAt)

        secondMigration := applied["2021-01-01 002"]
        if secondMigration == nil {
            t.Fatal("Missing second migration")
        } else if secondMigration.Checksum == "" {
            t.Fatal("Expected checksum to get populated when migration ran")
        }

        if firstMigration.AppliedAt.After(secondMigration.AppliedAt) {
            t.Errorf("Expected migrations to run in lexical order, but first migration
ran at %s and second one ran at %s", firstMigration.AppliedAt,
secondMigration.AppliedAt)
        }
    })
}

// TestFailedMigration ensures that a migration with a syntax error triggers
// an expected error when Apply() is run. This test is run on every dialect
// and every test database instance
func TestFailedMigration(t *testing.T) {
    withEachTestDB(t, func(t *testing.T, tdb *TestDB) {

        db := tdb.Connect(t)
        defer func() { _ = db.Close() }()

        tableName := time.Now().Format(time.RFC3339Nano)
        migrator := NewMigrator(WithTableName(tableName), WithDialect(tdb.Dialect))
        migrations := []*Migration{
            {
                ID:     "2019-01-01 Bad Migration",
                Script: "CREATE TIBBLE bad_table_name (id INTEGER NOT NULL PRIMARY
KEY)",
            },
        }
        err := migrator.Apply(db, migrations)
        expectErrorContains(t, err, "TIBBLE")
```

```
        query := "SELECT * FROM " + migrator.QuotedTableName()
        rows, _ := db.Query(query)

        // We expect either an error (because the transaction was rolled back
        // and the table no longer exists)... or  a query with no results
        if rows != nil {
            if rows.Next() {
                t.Error("Record was inserted in tracking table even though the
migration failed")
            }
            _ = rows.Close()
        }
    })

}

// TestSimultaneousApply creates multiple Migrators and multiple distinct
// connections to each test database and attempts to call .Apply() on them all
// concurrently. The migrations include an INSERT statement, which allows us
// to count to ensure that each unique migration was only run once.
func TestSimultaneousApply(t *testing.T) {
    concurrency := 4
    dataTable := fmt.Sprintf("data%d", rand.Int()) // #nosec we don't need
cryptographic security here
    migrationsTable := fmt.Sprintf("Migrations %s",
time.Now().Format(time.RFC3339Nano))
    sharedMigrations := []*Migration{
        {
            ID:     "2020-05-02 Create Data Table",
            Script: fmt.Sprintf(`CREATE TABLE %s (number INTEGER)`, dataTable),
        },
        {
            ID:     "2020-05-03 Add Initial Record",
            Script: fmt.Sprintf(`INSERT INTO %s (number) VALUES (1)`, dataTable),
        },
    }

    withEachTestDB(t, func(t *testing.T, tdb *TestDB) {
        var wg sync.WaitGroup
        for i := 0; i < concurrency; i++ {
            wg.Add(1)
            go func(i int) {
                db := tdb.Connect(t)
                defer func() { _ = db.Close() }()

                migrator := NewMigrator(WithDialect(tdb.Dialect),
WithTableName(migrationsTable))
                err := migrator.Apply(db, sharedMigrations)
                if err != nil {
                    t.Error(err)
                }
                _, err = db.Exec(fmt.Sprintf("INSERT INTO %s (number) VALUES (1)",
dataTable))
                if err != nil {
                    t.Error(err)
                }
                wg.Done()
            }(i)
        }
        wg.Wait()

        // We expect concurrency + 1 rows in the data table
        // (1 from the migration, and one each for the
```

```
        // goroutines which ran Apply and then did an
        // insert afterwards)
        db := tdb.Connect(t)
        defer func() { _ = db.Close() }()

        count := 0
        row := db.QueryRow(fmt.Sprintf("SELECT COUNT(*) FROM %s", dataTable))
        err := row.Scan(&count)
        if err != nil {
            t.Error(err)
        }
        if count != concurrency+1 {
            t.Errorf("Expected to get %d rows in %s table. Instead got %d",
concurrency+1, dataTable, count)
        }

    })
}

// TestMultiSchemaSupport ensures that each dialect and test database support
// having multiple tracking tables each tracking separate sets of migrations.
//
// The test scenario here is one set of "music" migrations which deal with
// artists, albums and tracks, and a separate set of "contacts" migrations
// which deal with contacts, phone_numbers, and addresses.
func TestMultiSchemaSupport(t *testing.T) {
    withEachTestDB(t, func(t *testing.T, tdb *TestDB) {
        music := NewMigrator(WithDialect(tdb.Dialect),
WithTableName("music_migrations"))
        contacts := NewMigrator(WithDialect(tdb.Dialect),
WithTableName("contacts_migrations"))

        // Use the same connection for both sets of migrations
        db := tdb.Connect(t)
        defer func() { _ = db.Close() }()

        // Apply the Music migrations
        err := music.Apply(db, testMigrations(t, "music"))
        if err != nil {
            t.Errorf("Failed to apply music migrations: %s", err)
        }

        // ... then the Contacts Migrations
        err = contacts.Apply(db, testMigrations(t, "contacts"))
        if err != nil {
            t.Errorf("Failed to apply contact migrations: %s", err)
        }

        // Then run a SELECT COUNT(*) query on each table to ensure that all of the
        // expected tables are co-existing in the same database and that they all
        // contain the expected number of rows (this approach is admittedly odd,
        // but it relies only on ANSI SQL code, so it should run on any SQL database).
        expectedRowCounts := map[string]int{
            "music_migrations":    3,
            "contacts_migrations": 3,
            "contacts":            1,
            "phone_numbers":       3,
            "addresses":           2,
            "artists":             0,
            "albums":              0,
            "tracks":              0,
        }
        for table, expectedRowCount := range expectedRowCounts {
            qtn := tdb.Dialect.QuotedTableName("", table)
```

```
            actualCount := -1 // Don't initialize to 0 because that's an expected value
            query := fmt.Sprintf("SELECT COUNT(*) FROM %s", qtn)
            rows, err := db.Query(query)
            if err != nil {
                t.Error(err)
            }
            if rows != nil && rows.Next() {
                err = rows.Scan(&actualCount)
                if err != nil {
                    t.Error(err)
                }
            } else {
                t.Errorf("Expected rows")
            }
            if actualCount != expectedRowCount {
                t.Errorf("Expected %d rows in table %s. Got %d", expectedRowCount, qtn,
actualCount)
            }
        }
    })
}

// TestRunFailure ensures that a low-level connection or query-related failure
// triggers an expected error.
func TestRunFailure(t *testing.T) {
    bq := BadQueryer{}
    m := makeTestMigrator()
    err := m.run(bq, testMigrations(t, "useless-ansi"))
    expectErrorContains(t, err, "SELECT id, checksum")

    err = m.run(nil, testMigrations(t, "useless-ansi"))
    if err != ErrNilDB {
        t.Errorf("Expected error '%s'. Got '%v'.", ErrNilDB, err)
    }
}

func TestNewMigratorApplyChain(t *testing.T) {
    // This is a compilability test... it is here to confirm that
    // NewMigrator()'s return value can have Apply() called on it.
    _ = NewMigrator().Apply(nil, testMigrations(t, "useless-ansi"))
}

// makeTestMigrator is a utility function which produces a migrator with an
// isolated environment (isolated due to a unique name for the migration
// tracking table).
func makeTestMigrator(options ...Option) *Migrator {
    tableName := time.Now().Format(time.RFC3339Nano)
    options = append(options, WithTableName(tableName))
    return NewMigrator(options...)
}

func testMigrations(t *testing.T, dirName string) []*Migration {
    path := fmt.Sprintf("test-migrations/%s", dirName)
    migrations, err := MigrationsFromDirectoryPath(path)
    if err != nil {
        t.Fatalf("Failed to load test migrations from '%s'", path)
    }
    return migrations
}

// assertZonesMatch accepts two Times and fails the test if their time zones
// don't match.
func assertZonesMatch(t *testing.T, expected, actual time.Time) {
    t.Helper()
```

```
    expectedName, expectedOffset := expected.Zone()
    actualName, actualOffset := actual.Zone()
    if expectedOffset != actualOffset {
        t.Errorf("Expected Zone '%s' with offset %d. Got Zone '%s' with offset %d",
expectedName, expectedOffset, actualName, actualOffset)
    }
}
}
```

mssql.go

```
package schema

import (
    "context"
    "fmt"
    "hash/crc32"
    "strings"
    "time"
    "unicode"
)

const mssqlAdvisoryLockSalt uint32 = 542384964

// MSSQL is the dialect for MS SQL-compatible databases
var MSSQL = mssqlDialect{}

type mssqlDialect struct{}

// Lock implements the Locker interface to obtain a global lock before the
// migrations are run. It uses SQL Server's sp_getapplock stored procedure
// with a session-based lock to ensure that only one process can run migrations
// at a time, which is critical for clustered environments.
func (s mssqlDialect) Lock(ctx context.Context, tx Queryer, tableName string) error {
    lockID := s.advisoryLockID(tableName)
    // Use application lock without explicit transaction
    query := fmt.Sprintf("EXEC sp_getapplock @Resource = '%d', @LockMode = 'Exclusive',
@LockOwner = 'Session';", lockID)
    _, err := tx.ExecContext(ctx, query)
    return err
}

// Unlock implements the Locker interface to release the global lock after the
// migrations are run. It first checks if we have the lock before trying to
// release it to avoid errors when the lock is not held.
func (s mssqlDialect) Unlock(ctx context.Context, tx Queryer, tableName string) error {
    lockID := s.advisoryLockID(tableName)

    // Check if we have the lock before trying to release it
    lockMode, err := s.getLockMode(ctx, tx, lockID)
    if err != nil {
        return err
    }

    // If we don't have a lock, nothing to release
    if lockMode == "NoLock" {
        return nil
    }

    // Release the application lock
```

```
    query := fmt.Sprintf("EXEC sp_releaseapplock @Resource = '%d', @LockOwner =
'Session';", lockID)
    _, err = tx.ExecContext(ctx, query)
    return err
}

// getLockMode checks if we currently hold a lock and returns the lock mode.
// This is extracted to a helper to ensure rows are properly closed before
// any subsequent queries on the same connection.
func (s mssqlDialect) getLockMode(ctx context.Context, tx Queryer, lockID uint32)
(string, error) {
    query := fmt.Sprintf("SELECT APPLOCK_MODE('public', '%d', 'Session');", lockID)
    rows, err := tx.QueryContext(ctx, query)
    if err != nil {
        return "", fmt.Errorf("error checking lock status: %w", err)
    }
    defer rows.Close()

    var lockMode string
    if rows.Next() {
        if err := rows.Scan(&lockMode); err != nil {
            return "", fmt.Errorf("error scanning lock mode: %w", err)
        }
    }
    return lockMode, nil
}

// advisoryLockID generates a consistent integer ID for use with SQL Server's
sp_getapplock
// based on the table name. It uses a CRC32 checksum of the table name XORed with a
salt
// to ensure uniqueness across different applications using the same database.
func (s mssqlDialect) advisoryLockID(tableName string) uint32 {
    return crc32.ChecksumIEEE([]byte(tableName)) ^ mssqlAdvisoryLockSalt
}

func (s mssqlDialect) QuotedTableName(schemaName, tableName string) string {
    if schemaName == "" {
        return s.QuotedIdent(tableName)
    }
    return fmt.Sprintf("%s.%s", s.QuotedIdent(schemaName), s.QuotedIdent(tableName))
}

func (s mssqlDialect) QuotedIdent(ident string) string {
    if ident == "" {
        return ""
    }

    var sb strings.Builder
    sb.WriteRune('[')
    for _, r := range ident {
        switch {
        case unicode.IsSpace(r):
            continue
        case r == ';':
            continue
        case r == ']':
            sb.WriteRune(r)
            sb.WriteRune(r)
        default:
            sb.WriteRune(r)
        }
    }
    sb.WriteRune(']')
```

```
    return sb.String()
}

func (s mssqlDialect) CreateMigrationsTable(ctx context.Context, tx Queryer, tableName
string) error {
    unquotedTableName := tableName[1 : len(tableName)-1]
    query := fmt.Sprintf(`
        IF NOT EXISTS (SELECT * FROM Sysobjects WHERE NAME='%s' AND XTYPE='U')
            CREATE TABLE %s (
                id VARCHAR(255) NOT NULL,
                checksum VARCHAR(32) NOT NULL DEFAULT '',
                execution_time_in_millis INTEGER NOT NULL DEFAULT 0,
                applied_at DATETIMEOFFSET NOT NULL
            )
    `, unquotedTableName, tableName)
    _, err := tx.ExecContext(ctx, query)

    // Handle concurrent table creation: ignore "object already exists" errors
    if err != nil && strings.Contains(err.Error(), "There is already an object named")
{
        return nil
    }
    return err
}

func (s mssqlDialect) GetAppliedMigrations(ctx context.Context, tx Queryer, tableName
string) (migrations []*AppliedMigration, err error) {
    migrations = make([]*AppliedMigration, 0)

    query := fmt.Sprintf(`
        SELECT id, checksum, execution_time_in_millis, applied_at
        FROM %s ORDER BY id ASC
    `, tableName)

    rows, err := tx.QueryContext(ctx, query)
    if err != nil {
        return migrations, err
    }
    defer rows.Close()

    for rows.Next() {
        migration := AppliedMigration{}
        err = rows.Scan(&migration.ID, &migration.Checksum,
&migration.ExecutionTimeInMillis, &migration.AppliedAt)
        if err != nil {
            err = fmt.Errorf("failed to GetAppliedMigrations. Did somebody change the
structure of the %s table?: %w", tableName, err)
            return migrations, err
        }
        migration.AppliedAt = migration.AppliedAt.In(time.Local)
        migrations = append(migrations, &migration)
    }

    return migrations, err
}

func (s mssqlDialect) InsertAppliedMigration(ctx context.Context, tx Queryer, tableName
string, am *AppliedMigration) error {
    query := fmt.Sprintf(`
        INSERT INTO %s
        ( id, checksum, execution_time_in_millis, applied_at )
        VALUES
        ( @p1, @p2, @p3, @p4 )`,
```

```
        tableName,
    )
    _, err := tx.ExecContext(ctx, query, am.ID, am.MD5(), am.ExecutionTimeInMillis,
am.AppliedAt)
    return err
}
}
```

mssql_test.go

```
package schema

import (
    "context"
    "database/sql"
    "fmt"
    "strings"
    "testing"

    sqlmock "github.com/DATA-DOG/go-sqlmock"
    // MSSQL Driver
    _ "github.com/microsoft/go-mssqldb"
)

// Interface verification that MSSQL is a valid Dialect
var (
    _ Dialect = MSSQL
)

func TestMSSQLQuotedTableName(t *testing.T) {
    type qtnTest struct {
        schema, table string
        expected      string
    }

    tests := []qtnTest{
        {"public", "ta[ble", `[public].[ta[ble]`},
        {"pu[b]lic", "ta[ble", `[pu[b]]lic].[ta[ble]`},
        {"tempdb", "users", `[tempdb].[users]`},
        {"schema.with.dot", "table.with.dot", `[schema.with.dot].[table.with.dot]`},
        {`public"`, `"; DROP TABLE users`, `[public"].["DROPTABLEusers]`},
    }

    for _, tc := range tests {
        tc := tc
        t.Run(tc.expected, func(t *testing.T) {
            actual := MSSQL.QuotedTableName(tc.schema, tc.table)
            if actual != tc.expected {
                t.Errorf("Expected %s, got %s", tc.expected, actual)
            }
        })
    }

}

func TestMSSQLQuotedIdent(t *testing.T) {
    table := map[string]string{
        "":                      "",
        "MY_TABLE":              "[MY_TABLE]",
        "users_roles":           `[users_roles]`,
```

```
        "table.with.dot":      `[table.with.dot]`,
        `table"with"quotes`:   `[table"with"quotes]`,
        "table[with]brackets": "[table[with]]brackets]",
    }

    for ident, expected := range table {
        t.Run(expected, func(t *testing.T) {
            actual := MSSQL.QuotedIdent(ident)
            if expected != actual {
                t.Errorf("Expected %s, got %s", expected, actual)
            }
        })
    }
}

// mssqlBadQueryer implements the Queryer interface but fails on specific queries
type mssqlBadQueryer struct {
    failOnQuery string
    scanError   bool
}

func (bq mssqlBadQueryer) ExecContext(ctx context.Context, query string, args
...interface{}) (sql.Result, error) {
    return nil, nil
}

func (bq mssqlBadQueryer) QueryContext(ctx context.Context, query string, args
...interface{}) (*sql.Rows, error) {
    if strings.Contains(query, bq.failOnQuery) {
        return nil, fmt.Errorf("query failed: %s", query)
    }
    return nil, nil
}

func TestMSSQLUnlockQueryError(t *testing.T) {
    bq := mssqlBadQueryer{failOnQuery: "APPLOCK_MODE"}
    err := MSSQL.Unlock(context.Background(), bq, "test_table")
    if err == nil {
        t.Error("Expected error, got nil")
    }
    if !strings.Contains(err.Error(), "error checking lock status") {
        t.Errorf("Expected error about lock status, got: %s", err)
    }
}

func TestMSSQLGetLockModeScanError(t *testing.T) {
    // Test that getLockMode handles scan errors properly
    // We use sqlmock to return rows that will cause a scan error
    db, mock, err := sqlmock.New()
    if err != nil {
        t.Fatal(err)
    }
    defer db.Close()

    // Return a row with wrong type that will fail to scan into string
    rows := sqlmock.NewRows([]string{"lock_mode"}).AddRow(nil)
    mock.ExpectQuery("SELECT APPLOCK_MODE").WillReturnRows(rows)

    conn, err := db.Conn(context.Background())
    if err != nil {
        t.Fatal(err)
    }
    defer conn.Close()
```

```
    _, err = mssqlDialect{}.getLockMode(context.Background(), conn, 12345)
    if err == nil {
        t.Error("Expected error from scan, got nil")
    }
    if !strings.Contains(err.Error(), "error scanning lock mode") {
        t.Errorf("Expected scan error, got: %s", err)
    }
}

func TestMSSQLCreateMigrationsTableConcurrentError(t *testing.T) {
    // Test that concurrent table creation errors are ignored
    db, mock, err := sqlmock.New()
    if err != nil {
        t.Fatal(err)
    }
    defer db.Close()

    // Simulate the "object already exists" error from concurrent creation
    mock.ExpectExec("IF NOT EXISTS").WillReturnError(
        fmt.Errorf("There is already an object named 'schema_migrations' in the
database"),
    )

    conn, err := db.Conn(context.Background())
    if err != nil {
        t.Fatal(err)
    }
    defer conn.Close()

    err = mssqlDialect{}.CreateMigrationsTable(context.Background(), conn, "
[schema_migrations]")
    if err != nil {
        t.Errorf("Expected nil error for concurrent creation, got: %s", err)
    }
}
}
```

mysql.go

```
package schema

import (
    "context"
    "fmt"
    "hash/crc32"
    "strings"
    "time"
)

const mysqlLockSalt uint32 = 271192482

// MySQL is the dialect which should be used for MySQL/MariaDB databases
var MySQL = mysqlDialect{}

type mysqlDialect struct{}

// Lock implements the Locker interface to obtain a global lock before the
// migrations are run.
func (m mysqlDialect) Lock(ctx context.Context, tx Queryer, tableName string) error {
    lockID := m.advisoryLockID(tableName)
```

```go
    query := fmt.Sprintf(`SELECT GET_LOCK('%s', 10)`, lockID)
    _, err := tx.ExecContext(ctx, query)
    return err
}

// Unlock implements the Locker interface to release the global lock after the
// migrations are run.
func (m mysqlDialect) Unlock(ctx context.Context, tx Queryer, tableName string) error {
    lockID := m.advisoryLockID(tableName)
    query := fmt.Sprintf(`SELECT RELEASE_LOCK('%s')`, lockID)
    _, err := tx.ExecContext(ctx, query)
    return err
}

// CreateMigrationsTable implements the Dialect interface to create the
// table which tracks applied migrations. It only creates the table if it
// does not already exist
func (m mysqlDialect) CreateMigrationsTable(ctx context.Context, tx Queryer, tableName
string) error {
    query := fmt.Sprintf(`
        CREATE TABLE IF NOT EXISTS %s (
            id VARCHAR(255) NOT NULL,
            checksum VARCHAR(32) NOT NULL DEFAULT '',
            execution_time_in_millis INTEGER NOT NULL DEFAULT 0,
            applied_at TIMESTAMP NOT NULL
        )`, tableName)
    _, err := tx.ExecContext(ctx, query)
    return err
}

// InsertAppliedMigration implements the Dialect interface to insert a record
// into the migrations tracking table *after* a migration has successfully
// run.
func (m mysqlDialect) InsertAppliedMigration(ctx context.Context, tx Queryer, tableName
string, am *AppliedMigration) error {
    query := fmt.Sprintf(`
        INSERT INTO %s
        ( id, checksum, execution_time_in_millis, applied_at )
        VALUES
        ( ?, ?, ?, ? )
        `, tableName,
    )
    _, err := tx.ExecContext(ctx, query, am.ID, am.MD5(), am.ExecutionTimeInMillis,
am.AppliedAt)
    return err
}

// GetAppliedMigrations retrieves all data from the migrations tracking table
func (m mysqlDialect) GetAppliedMigrations(ctx context.Context, tx Queryer, tableName
string) (migrations []*AppliedMigration, err error) {
    migrations = make([]*AppliedMigration, 0)

    query := fmt.Sprintf(`
        SELECT id, checksum, execution_time_in_millis, applied_at
        FROM %s
        ORDER BY id ASC`, tableName)
    rows, err := tx.QueryContext(ctx, query)
    if err != nil {
        return migrations, err
    }
    defer rows.Close()

    for rows.Next() {
        migration := AppliedMigration{}
```

```
        var appliedAt mysqlTime
        err = rows.Scan(&migration.ID, &migration.Checksum,
&migration.ExecutionTimeInMillis, &appliedAt)
        if err != nil {
            err = fmt.Errorf("Failed to GetAppliedMigrations. Did somebody change the
structure of the %s table?: %w", tableName, err)
            return migrations, err
        }
        migration.AppliedAt = appliedAt.Value
        migrations = append(migrations, &migration)
    }

    return migrations, err
}

// QuotedTableName returns the string value of the name of the migration
// tracking table after it has been quoted for MySQL
func (m mysqlDialect) QuotedTableName(schemaName, tableName string) string {
    if schemaName == "" {
        return m.quotedIdent(tableName)
    }
    return m.quotedIdent(schemaName) + "." + m.quotedIdent(tableName)
}

// quotedIdent wraps the supplied string in the MySQL identifier
// quote character
func (m mysqlDialect) quotedIdent(ident string) string {
    if ident == "" {
        return ""
    }
    return "`" + strings.ReplaceAll(ident, "`", "``") + "`"
}

// advisoryLockID generates a table-specific lock name to use
func (m mysqlDialect) advisoryLockID(tableName string) string {
    sum := crc32.ChecksumIEEE([]byte(tableName))
    sum = sum * mysqlLockSalt
    return fmt.Sprint(sum)
}

type mysqlTime struct {
    Value time.Time
}

func (t *mysqlTime) Scan(src interface{}) (err error) {
    if src == nil {
        t.Value = time.Time{}
    }

    if srcTime, isTime := src.(time.Time); isTime {
        t.Value = srcTime.In(time.Local)
        return nil
    }

    return t.ScanString(fmt.Sprintf("%s", src))
}

func (t *mysqlTime) ScanString(src string) (err error) {
    switch len(src) {
    case 19:
        t.Value, err = time.ParseInLocation("2006-01-02 15:04:05", src, time.UTC)
        if err != nil {
            return err
```

```
        }
    }
    t.Value = t.Value.In(time.Local)
    return nil
}
}
```

mysql_test.go

```
package schema

import (
    "testing"
    "time"

    // MySQL Driver
    _ "github.com/go-sql-driver/mysql"
)

// Interface verification that MySQL is a valid Dialect
var (
    _ Dialect = MySQL
    _ Locker  = MySQL
)

func TestMySQLQuotedTableName(t *testing.T) {
    type qtnTest struct {
        schema, table string
        expected      string
    }

    table := []qtnTest{
        {"public", "users", "`public`.`users`"},
        {"schema.with.dot", "table.with.dot", "`schema.with.dot`.`table.with.dot`"},
        {"schema`with`tick", "table`with`tick",
"`schema``with``tick`.`table``with``tick`"},
    }

    for _, test := range table {
        actual := MySQL.QuotedTableName(test.schema, test.table)
        if actual != test.expected {
            t.Errorf("Expected %s, got %s", test.expected, actual)
        }
    }
}

func TestMySQLQuotedIdent(t *testing.T) {
    table := map[string]string{
        "":                  "",
        "MY_TABLE":          "`MY_TABLE`",
        "users_roles":       "`users_roles`",
        "table.with.dot":    "`table.with.dot`",
        `table"with"quotes`: "`table\"with\"quotes`",
        "table`with`ticks":  "`table``with``ticks`",
    }

    for input, expected := range table {
        actual := MySQL.quotedIdent(input)
        if actual != expected {
            t.Errorf("Expected %s, got %s", expected, actual)
```

```
        }
    }
}

func TestMySQLTimeScanner(t *testing.T) {
    t.Run("Nil", func(t *testing.T) {
        v := mysqlTime{}
        err := v.Scan(nil)
        if err != nil {
            t.Error(err)
        }
    })

    t.Run("Time In UTC", func(t *testing.T) {
        v := mysqlTime{}
        expected := time.Date(2021, 1, 1, 18, 19, 20, 0, time.UTC)
        src, _ := time.ParseInLocation("2006-01-02 15:04:05", "2021-01-01 18:19:20",
time.UTC)
        err := v.Scan(src)
        if err != nil {
            t.Error(err)
        }
        assertZonesMatch(t, time.Date(2021, 1, 1, 18, 19, 20, 0, time.Local), v.Value)
        if expected.Unix() != v.Value.Unix() {
            t.Errorf("Expected %s, got %s", expected.Format(time.RFC3339),
v.Value.Format(time.RFC3339))
        }
    })

    t.Run("Invalid String Time", func(t *testing.T) {
        v := mysqlTime{}
        err := v.Scan("2000-13-45 99:45:23")
        if err == nil {
            t.Errorf("Expected error scanning invalid time")
        }
    })
}

type nullMySQLLogger struct{}

func (nsl nullMySQLLogger) Print(v ...interface{}) {
    // Intentional no-op. The purpose of this class is to swallow/ignore
    // the MySQL driver errors which occur while we're waiting for the Docker
    // MySQL instance to start up.
}
}
```

options.go

```
package schema

import "context"

// Option supports option chaining when creating a Migrator.
// An Option is a function which takes a Migrator and
// returns a Migrator with an Option modified.
type Option func(m Migrator) Migrator

// WithDialect builds an Option which will set the supplied
// dialect on a Migrator. Usage: NewMigrator(WithDialect(MySQL))
```

```
func WithDialect(dialect Dialect) Option {
    return func(m Migrator) Migrator {
        m.Dialect = dialect
        return m
    }
}

// WithTableName is an option which customizes the name of the schema_migrations
// tracking table. It can be called with either 1 or 2 string arguments. If
// called with 2 arguments, the first argument is assumed to be a schema
// qualifier (for example, WithTableName("public", "schema_migrations") would
// assign the table named "schema_migrations" in the the default "public"
// schema for Postgres)
func WithTableName(names ...string) Option {
    return func(m Migrator) Migrator {
        switch len(names) {
        case 0:
            // No-op if no customization was provided
        case 1:
            m.TableName = names[0]
        default:
            m.SchemaName = names[0]
            m.TableName = names[1]
        }
        return m
    }
}

// WithContext is an Option which sets the Migrator to run within the provided
// Context
func WithContext(ctx context.Context) Option {
    return func(m Migrator) Migrator {
        m.ctx = ctx
        return m
    }
}

// Logger is the interface for logging operations of the logger.
// By default the migrator operates silently. Providing a Logger
// enables output of the migrator's operations.
type Logger interface {
    Print(...interface{})
}

// WithLogger builds an Option which will set the supplied Logger
// on a Migrator. Usage: NewMigrator(WithLogger(logrus.New()))
func WithLogger(logger Logger) Option {
    return func(m Migrator) Migrator {
        m.Logger = logger
        return m
    }
}
}
```

options_test.go

```
package schema

import (
    "context"
```

```
    "fmt"
    "log"
    "os"
    "strings"
    "testing"
)

func TestWithTableNameOptionWithSchema(t *testing.T) {
    schema := "special"
    table := "my_migrations"
    m := NewMigrator(WithTableName(schema, table))
    if m.SchemaName != schema {
        t.Errorf("Expected SchemaName to be '%s'. Got '%s' instead.", schema,
m.SchemaName)
    }
    if m.TableName != table {
        t.Errorf("Expected TableName to be '%s'. Got '%s' instead.", table,
m.TableName)
    }
}
func TestWithTableNameOptionWithoutSchema(t *testing.T) {
    name := "terrible_migrations_table_name"
    m := NewMigrator(WithTableName(name))
    if m.SchemaName != "" {
        t.Errorf("Expected SchemaName to be blank. Got '%s' instead.", m.SchemaName)
    }
    if m.TableName != name {
        t.Errorf("Expected TableName to be '%s'. Got '%s' instead.", name, m.TableName)
    }
}

func TestDefaultTableName(t *testing.T) {
    name := "schema_migrations"
    m := NewMigrator()
    if m.SchemaName != "" {
        t.Errorf("Expected SchemaName to be blank by default. Got '%s' instead.",
m.SchemaName)
    }
    if m.TableName != name {
        t.Errorf("Expected TableName to be '%s' by default. Got '%s' instead.", name,
m.TableName)
    }
}

func TestDefaultDialect(t *testing.T) {
    m := NewMigrator()
    if m.Dialect != Postgres {
        t.Errorf("Expected Migrator to have Postgres Dialect by default. Got: %v",
m.Dialect)
    }
}

func TestWithDialectOption(t *testing.T) {
    m := Migrator{Dialect: nil}
    if m.Dialect != nil {
        t.Errorf("Expected nil Dialect. Got '%v'", m.Dialect)
    }
    modifiedMigrator := WithDialect(Postgres)(m)
    if modifiedMigrator.Dialect != Postgres {
        t.Errorf("Expected modifiedMigrator to have Postgres dialect. Got '%v'.",
modifiedMigrator.Dialect)
    }
    if m.Dialect != nil {
        t.Errorf("Expected Option to not modify the original Migrator's Dialect, but it
```

```
changed it to '%v'.", m.Dialect)
    }
}

func TestWithContextOption(t *testing.T) {
    m := NewMigrator()
    if m.ctx == nil {
        t.Errorf("Expected NewMigrator to set a default .ctx")
    }
    ctx := context.WithValue(context.Background(), "key", "value")
    m = NewMigrator(WithContext(ctx))
    if m.ctx != ctx {
        t.Errorf("Expected WithContext to set a custom context")
    }
}

func TestWithLoggerOption(t *testing.T) {
    m := Migrator{}
    if m.Logger != nil {
        t.Errorf("Expected nil Logger by default. Got '%v'", m.Logger)
    }
    modifiedMigrator := WithLogger(log.New(os.Stdout, "schema: ", log.Ldate|log.Ltime))
(m)
    if modifiedMigrator.Logger == nil {
        t.Errorf("Expected logger to have been added")
    }
}

type StrLog string

func (nl *StrLog) Print(msgs ...interface{}) {
    var sb strings.Builder
    for _, msg := range msgs {
        sb.WriteString(fmt.Sprintf("%s", msg))
    }
    result := StrLog(sb.String())
    *nl = result
}

func TestSimpleLogger(t *testing.T) {
    var str StrLog
    m := NewMigrator(WithLogger(&str))
    m.log("Test message")
    if str != "Test message" {
        t.Errorf("Expected logger to print 'Test message'. Got '%s'", str)
    }
}
}
```

postgres.go

```
package schema

import (
    "context"
    "fmt"
    "hash/crc32"
    "strings"
    "time"
    "unicode"
```

```
)

const postgresAdvisoryLockSalt uint32 = 542384964

// Postgres is the dialect for Postgres-compatible
// databases
var Postgres = postgresDialect{}

type postgresDialect struct{}

// Lock implements the Locker interface to obtain a global lock before the
// migrations are run.
func (p postgresDialect) Lock(ctx context.Context, tx Queryer, tableName string) error
{
    lockID := p.advisoryLockID(tableName)
    query := fmt.Sprintf("SELECT pg_advisory_lock(%s)", lockID)
    _, err := tx.ExecContext(ctx, query)
    return err
}

// Unlock implements the Locker interface to release the global lock after the
// migrations are run.
func (p postgresDialect) Unlock(ctx context.Context, tx Queryer, tableName string)
error {
    lockID := p.advisoryLockID(tableName)
    query := fmt.Sprintf("SELECT pg_advisory_unlock(%s)", lockID)
    _, err := tx.ExecContext(ctx, query)
    return err
}

// CreateMigrationsTable implements the Dialect interface to create the
// table which tracks applied migrations. It only creates the table if it
// does not already exist
func (p postgresDialect) CreateMigrationsTable(ctx context.Context, tx Queryer,
tableName string) error {
    query := fmt.Sprintf(`
                CREATE TABLE IF NOT EXISTS %s (
                    id VARCHAR(255) NOT NULL,
                    checksum VARCHAR(32) NOT NULL DEFAULT '',
                    execution_time_in_millis INTEGER NOT NULL DEFAULT 0,
                    applied_at TIMESTAMP WITH TIME ZONE NOT NULL
                )
            `, tableName)
    _, err := tx.ExecContext(ctx, query)
    return err
}

// InsertAppliedMigration implements the Dialect interface to insert a record
// into the migrations tracking table *after* a migration has successfully
// run.
func (p postgresDialect) InsertAppliedMigration(ctx context.Context, tx Queryer,
tableName string, am *AppliedMigration) error {
    query := fmt.Sprintf(`
        INSERT INTO %s
        ( id, checksum, execution_time_in_millis, applied_at )
        VALUES
        ( $1, $2, $3, $4 )`,
        tableName,
    )
    _, err := tx.ExecContext(ctx, query, am.ID, am.MD5(), am.ExecutionTimeInMillis,
am.AppliedAt)
    return err
}
```

```go
// GetAppliedMigrations retrieves all data from the migrations tracking table
func (p postgresDialect) GetAppliedMigrations(ctx context.Context, tx Queryer,
tableName string) (migrations []*AppliedMigration, err error) {
    migrations = make([]*AppliedMigration, 0)

    query := fmt.Sprintf(`
        SELECT id, checksum, execution_time_in_millis, applied_at
        FROM %s ORDER BY id ASC
    `, tableName)
    rows, err := tx.QueryContext(ctx, query)
    if err != nil {
        return migrations, err
    }
    defer rows.Close()

    for rows.Next() {
        migration := AppliedMigration{}
        err = rows.Scan(&migration.ID, &migration.Checksum,
&migration.ExecutionTimeInMillis, &migration.AppliedAt)
        if err != nil {
            err = fmt.Errorf("failed to GetAppliedMigrations. Did somebody change the
structure of the %s table?: %w", tableName, err)
            return migrations, err
        }
        migration.AppliedAt = migration.AppliedAt.In(time.Local)
        migrations = append(migrations, &migration)
    }

    return migrations, err
}

// QuotedTableName returns the string value of the name of the migration
// tracking table after it has been quoted for Postgres
func (p postgresDialect) QuotedTableName(schemaName, tableName string) string {
    if schemaName == "" {
        return p.QuotedIdent(tableName)
    }
    return p.QuotedIdent(schemaName) + "." + p.QuotedIdent(tableName)
}

// QuotedIdent wraps the supplied string in the Postgres identifier
// quote character
func (p postgresDialect) QuotedIdent(ident string) string {
    if ident == "" {
        return ""
    }

    var sb strings.Builder
    sb.WriteRune('"')
    for _, r := range ident {
        switch {
        case unicode.IsSpace(r):
            // Skip spaces
            continue
        case r == '"':
            // Escape double-quotes with repeated double-quotes
            sb.WriteString(`""`)
        case r == ';':
            // Ignore the command termination character
            continue
        default:
            sb.WriteRune(r)
        }
    }
```

```
    sb.WriteRune('"')
    return sb.String()
}

// advisoryLockID generates a table-specific lock name to use
func (p postgresDialect) advisoryLockID(tableName string) string {
    sum := crc32.ChecksumIEEE([]byte(tableName))
    sum = sum * postgresAdvisoryLockSalt
    return fmt.Sprint(sum)
}
}
```

postgres_test.go

```
package schema

import (
    "testing"

    // Postgres Driver
    _ "github.com/lib/pq"
)

// Interface verification that Postgres is a valid Dialect
var (
    _ Dialect = Postgres
    _ Locker  = Postgres
)

func TestPostgreSQLQuotedTableName(t *testing.T) {
    type qtnTest struct {
        schema, table string
        expected      string
    }
    tests := []qtnTest{
        {"public", "users", `"public"."users"`},
        {"schema.with.dot", "table.with.dot", `"schema.with.dot"."table.with.dot"`},
        {`public"`, `"; DROP TABLE users`, `"public""."""DROPTABLEusers"`},
    }
    for _, test := range tests {
        actual := Postgres.QuotedTableName(test.schema, test.table)
        if actual != test.expected {
            t.Errorf("Expected %s, got %s", test.expected, actual)
        }
    }
}

func TestPostgreSQLQuotedIdent(t *testing.T) {
    table := map[string]string{
        "":                  "",
        "MY_TABLE":          `"MY_TABLE"`,
        "users_roles":       `"users_roles"`,
        "table.with.dot":    `"table.with.dot"`,
        `table"with"quotes`: `"table""with""quotes"`,
    }
    for ident, expected := range table {
        actual := Postgres.QuotedIdent(ident)
        if expected != actual {
            t.Errorf("Expected %s, got %s", expected, actual)
        }
```

```
    }
}
}
```

schema.go

```
package schema

import (
    "context"
    "database/sql"
    "errors"
)

// DefaultTableName defines the name of the database table which will
// hold the status of applied migrations
const DefaultTableName = "schema_migrations"

// ErrNilDB is thrown when the database pointer is nil
var ErrNilDB = errors.New("DB pointer is nil")

// DB  defines the interface for a *sql.DB, which can be used to get a concrete
// connection to the database.
type DB interface {
    Conn(ctx context.Context) (*sql.Conn, error)
}

// Connection defines the interface for a *sql.Conn, which can both start a new
// transaction and run queries.
type Connection interface {
    Transactor
    Queryer
}

// Queryer is something which can execute a Query (either a sql.DB
// or a sql.Tx)
type Queryer interface {
    ExecContext(ctx context.Context, query string, args ...interface{}) (sql.Result,
error)
    QueryContext(ctx context.Context, query string, args ...interface{}) (*sql.Rows,
error)
}

// Transactor defines the interface for the Begin method from the *sql.DB
type Transactor interface {
    BeginTx(ctx context.Context, opts *sql.TxOptions) (*sql.Tx, error)
}
}
```

schema_test.go

```
package schema

import (
    "database/sql"
```

```
)

// Interface verification that *sql.DB satisfies our Connection interface
var (
    _ Connection = &sql.DB{}
)

// Interface verification that *sql.DB satisfies our Transactor interface
var (
    _ Transactor = &sql.DB{}
)

// Interface verification that *sql.Tx and *sql.DB both satisfy our
// Queryer interface
var (
    _ Queryer = &sql.DB{}
    _ Queryer = &sql.Tx{}
)

const (
    PostgresDriverName = "postgres"
    SQLiteDriverName   = "sqlite3"
    MySQLDriverName    = "mysql"
    MSSQLDriverName    = "sqlserver"
)

// TestDBs holds all of the specific database instances against which tests
// will run. The connectDB test helper references the keys of this map, and
// the withEachDB helper runs tests against every database defined here.
var TestDBs map[string]*TestDB = map[string]*TestDB{
    "postgres:latest": {
        Dialect:    Postgres,
        Driver:     PostgresDriverName,
        DockerRepo: "postgres",
        DockerTag:  "latest",
    },
    "sqlite": {
        Dialect: SQLite,
        Driver:  SQLiteDriverName,
    },
    "mysql:latest": {
        Dialect:    MySQL,
        Driver:     MySQLDriverName,
        DockerRepo: "mysql/mysql-server",
        DockerTag:  "latest",
    },
    "mariadb:latest": {
        Dialect:    MySQL,
        Driver:     MySQLDriverName,
        DockerRepo: "mariadb",
        DockerTag:  "latest",
    },
    "mssql:latest": {
        Dialect:    MSSQL,
        Driver:     MSSQLDriverName,
        DockerRepo: "mcr.microsoft.com/mssql/server",
        DockerTag:  "2019-latest",
        // SkippedArchs: []string{"amd64"},
    },
}
}
```

sqlite.go

```go
package schema

import (
    "context"
    "fmt"
    "strings"
    "time"
    "unicode"
)

// SQLite is the dialect for sqlite3 databases
var SQLite = &sqliteDialect{}

type sqliteDialect struct{}

// CreateMigrationsTable implements the Dialect interface to create the
// table which tracks applied migrations. It only creates the table if it
// does not already exist
func (s sqliteDialect) CreateMigrationsTable(ctx context.Context, tx Queryer, tableName
string) error {
    query := fmt.Sprintf(`
        CREATE TABLE IF NOT EXISTS %s (
            id TEXT NOT NULL,
            checksum TEXT NOT NULL DEFAULT '',
            execution_time_in_millis INTEGER NOT NULL DEFAULT 0,
            applied_at DATETIME NOT NULL
        )`, tableName)
    _, err := tx.ExecContext(ctx, query)
    return err
}

// InsertAppliedMigration implements the Dialect interface to insert a record
// into the migrations tracking table *after* a migration has successfully
// run.
func (s *sqliteDialect) InsertAppliedMigration(ctx context.Context, tx Queryer,
tableName string, am *AppliedMigration) error {
    query := fmt.Sprintf(`
        INSERT INTO %s
        ( id, checksum, execution_time_in_millis, applied_at )
        VALUES
        ( ?, ?, ?, ? )
        `, tableName,
    )
    _, err := tx.ExecContext(ctx, query, am.ID, am.MD5(), am.ExecutionTimeInMillis,
am.AppliedAt)
    return err
}

// GetAppliedMigrations retrieves all data from the migrations tracking table
func (s sqliteDialect) GetAppliedMigrations(ctx context.Context, tx Queryer, tableName
string) (migrations []*AppliedMigration, err error) {
    migrations = make([]*AppliedMigration, 0)

    query := fmt.Sprintf(`
        SELECT id, checksum, execution_time_in_millis, applied_at
        FROM %s
        ORDER BY id ASC
    `, tableName)
    rows, err := tx.QueryContext(ctx, query)
    if err != nil {
```

```
        return migrations, err
    }
    defer rows.Close()

    for rows.Next() {
        migration := AppliedMigration{}
        err = rows.Scan(&migration.ID, &migration.Checksum,
&migration.ExecutionTimeInMillis, &migration.AppliedAt)
        if err != nil {
            err = fmt.Errorf("Failed to GetAppliedMigrations. Did somebody change the
structure of the %s table?: %w", tableName, err)
            return migrations, err
        }
        migration.AppliedAt = migration.AppliedAt.In(time.Local)
        migrations = append(migrations, &migration)
    }

    return migrations, err
}

// QuotedTableName returns the string value of the name of the migration
// tracking table after it has been quoted for SQLite
func (s sqliteDialect) QuotedTableName(schemaName, tableName string) string {
    ident := schemaName + tableName
    if ident == "" {
        return ""
    }

    var sb strings.Builder
    sb.WriteRune('"')
    for _, r := range ident {
        switch {
        case unicode.IsSpace(r):
            // Skip spaces
            continue
        case r == '"':
            // Escape double-quotes with repeated double-quotes
            sb.WriteString(`""`)
        case r == ';':
            // Ignore the command termination character
            continue
        default:
            sb.WriteRune(r)
        }
    }
    sb.WriteRune('"')
    return sb.String()

}
}
```

sqlite_test.go

```
package schema

import (
    "testing"

    // SQLite driver
    _ "github.com/mattn/go-sqlite3"
```

```
)

// Interface verification that SQLite is a valid Dialect
var (
    _ Dialect = SQLite
)

func TestSQLiteQuotedTableName(t *testing.T) {
    table := map[string]string{
        "":                      "",
        "MY_TABLE":              `"MY_TABLE"`,
        "users_roles":           `"users_roles"`,
        "table.with.dot":        `"table.with.dot"`,
        `table"with"quotes`:    `"table""with""quotes"`,
        `"; DROP TABLE users`: `"""DROPTABLEusers"`,
    }
    for ident, expected := range table {
        actual := SQLite.QuotedTableName("", ident)
        if expected != actual {
            t.Errorf("Expected %s, got %s", expected, actual)
        }
        actual = SQLite.QuotedTableName(ident, "")
        if expected != actual {
            t.Errorf("Expected %s, got %s when table came first", expected, actual)
        }
    }
}
}
```

Chapter 1.0.0

test-migrations

Chapter 1.1.0

test-migrations/contacts

test-migrations/contacts/0000-00-00 001 Contacts.sql

```
CREATE TABLE contacts (id INTEGER PRIMARY KEY);
INSERT INTO contacts (id) VALUES (1);}
```

test-migrations/contacts/0000-00-00 002 Phone Numbers.sql

```
CREATE TABLE phone_numbers (
    id INTEGER PRIMARY KEY,
    contact_id INTEGER
);
INSERT INTO phone_numbers (id, contact_id) VALUES (1, 1);
INSERT INTO phone_numbers (id, contact_id) VALUES (2, 1);
INSERT INTO phone_numbers (id, contact_id) VALUES (3, 1);}
```

test-migrations/contacts/0000-00-00 003 Addresses.sql

```
CREATE TABLE addresses (
    id INTEGER PRIMARY KEY,
    contact_id INTEGER
);
INSERT INTO addresses (id, contact_id) VALUES (1,1);
INSERT INTO addresses (id, contact_id) VALUES (2, 1);}
```

Chapter 1.2.0

test-migrations/music

test-migrations/music/0000-00-00 001 Artists.sql

```
CREATE TABLE artists (id INTEGER PRIMARY KEY);}
```

test-migrations/music/0000-00-00 002 Albums.sql

```
CREATE TABLE albums (
    id INTEGER PRIMARY KEY,
    artist_id INTEGER
);}
```

test-migrations/music/0000-00-00 003 Tracks.sql

```
CREATE TABLE tracks (
    id INTEGER PRIMARY KEY,
    artist_id INTEGER,
```

```
    album_id INTEGER
);}
```

Chapter 1.3.0

test-migrations/saas

test-migrations/saas/2019-01-01 0900 Create Users.sql

```
CREATE TABLE users (id INTEGER NOT NULL PRIMARY KEY);}
```

test-migrations/saas/2019-01-03 1000 Create Affiliates.sql

```
CREATE TABLE affiliates (id INTEGER NOT NULL PRIMARY KEY);}
```

Chapter 1.4.0

test-migrations/unreadable

test-migrations/unreadable/readable.sql

```
CREATE fake_table (id INTEGER);}
```

test-migrations/unreadable/unreadable.sql

```
CREATE TABLE fake (id INTEGER);}
```

Chapter 1.5.0

test-migrations/useless-ansi

test-migrations/useless-ansi/0000-00-00 001 Select 1.sql

```
SELECT 1;}
```

test-migrations/useless-ansi/0000-00-00 002 Select 2.sql

```
SELECT 2;}
```

testdb_test.go

```go
package schema

import (
    "database/sql"
    "fmt"
    "log"
    "os"
    "runtime"
    "testing"

    dockertest "github.com/ory/dockertest/v3"
    "github.com/ory/dockertest/v3/docker"
)

// TestDB represents a specific database instance against which we would like
// to run database migration tests.
type TestDB struct {
    Dialect      Dialect
    Driver       string
    DockerRepo   string
    DockerTag    string
    Resource     *dockertest.Resource
    SkippedArchs []string
    path         string
}

func (c *TestDB) Username() string {
    switch c.Driver {
    case MSSQLDriverName:
        return "SA"
    default:
        return "schemauser"
    }
}

func (c *TestDB) Password() string {
    switch c.Driver {
```

```
    case MSSQLDriverName:
        return "Th1sI5AMor3_Compl1c4tedPasswd!"
    default:
        return "schemasecret"
    }
}

func (c *TestDB) DatabaseName() string {
    switch c.Driver {
    case MSSQLDriverName:
        return "master"
    default:
        return "schematests"
    }
}

// Port asks Docker for the host-side port we can use to connect to the
// relevant container's database port.
func (c *TestDB) Port() string {
    switch c.Driver {
    case MySQLDriverName:
        return c.Resource.GetPort("3306/tcp")
    case PostgresDriverName:
        return c.Resource.GetPort("5432/tcp")
    case MSSQLDriverName:
        return c.Resource.GetPort("1433/tcp")
    }
    return ""
}

func (c *TestDB) IsDocker() bool {
    return c.DockerRepo != "" && c.DockerTag != ""
}

func (c *TestDB) IsSQLite() bool {
    return c.Driver == SQLiteDriverName
}

// DockerEnvars computes the environment variables that are needed for a
// docker instance.
func (c *TestDB) DockerEnvars() []string {
    switch c.Driver {
    case PostgresDriverName:
        return []string{
            fmt.Sprintf("POSTGRES_USER=%s", c.Username()),
            fmt.Sprintf("POSTGRES_PASSWORD=%s", c.Password()),
            fmt.Sprintf("POSTGRES_DB=%s", c.DatabaseName()),
        }
    case MySQLDriverName:
        return []string{
            "MYSQL_RANDOM_ROOT_PASSWORD=true",
            fmt.Sprintf("MYSQL_USER=%s", c.Username()),
            fmt.Sprintf("MYSQL_PASSWORD=%s", c.Password()),
            fmt.Sprintf("MYSQL_DATABASE=%s", c.DatabaseName()),
        }
    case MSSQLDriverName:
        return []string{
            "ACCEPT_EULA=Y",
            fmt.Sprintf("SA_USER=%s", c.Username()),
            fmt.Sprintf("SA_PASSWORD=%s", c.Password()),
            fmt.Sprintf("SA_DATABASE=%s", c.DatabaseName()),
        }
    default:
        return []string{}
```

```
    }
}

func (c *TestDB) IsRunnable() bool {
    for _, skippedArch := range c.SkippedArchs {
        if skippedArch == runtime.GOARCH {
            return false
        }
    }
    return true
}

// Path computes the full path to the database on disk (applies only to SQLite
// instances).
func (c *TestDB) Path() string {
    switch c.Driver {
    case SQLiteDriverName:
        if c.path == "" {
            tmpF, _ := os.CreateTemp("", "schema.*.sqlite3")
            c.path = tmpF.Name()
        }
        return c.path
    default:
        return ""
    }
}

func (c *TestDB) DSN() string {
    switch c.Driver {
    case PostgresDriverName:
        return fmt.Sprintf("postgres://%s:%s@localhost:%s/%s?sslmode=disable",
c.Username(), c.Password(), c.Port(), c.DatabaseName())
    case SQLiteDriverName:
        return c.Path()
    case MySQLDriverName:
        /**
         * Since we want the system to be compatible with both parseTime=true and
         * not, we use different querystrings with MariaDB and MySQL.
         */
        if c.DockerRepo == "mariadb" {
            return fmt.Sprintf("%s:%s@(localhost:%s)/%s?
parseTime=true&multiStatements=true", c.Username(), c.Password(), c.Port(),
c.DatabaseName())
        }
        return fmt.Sprintf("%s:%s@(localhost:%s)/%s?multiStatements=true",
c.Username(), c.Password(), c.Port(), c.DatabaseName())
    case MSSQLDriverName:
        return fmt.Sprintf("sqlserver://%s:%s@localhost:%s/?
database=%s&encrypt=disable", c.Username(), c.Password(), c.Port(), c.DatabaseName())
    }
    // TODO Return error
    return "NoDSN"
}

// Init sets up a test database instance for connections. For dockertest-based
// instances, this function triggers the `docker run` call. For SQLite-based
// test instances, this creates the data file. In all cases, we verify that
// the database is connectable via a test connection.
func (c *TestDB) Init(pool *dockertest.Pool) {
    var err error

    if c.IsDocker() {
        // For Docker-based test databases, we send a startup signal to have Docker
        // launch a container for this test run.
```

```
        log.Printf("Starting docker container %s:%s\n", c.DockerRepo, c.DockerTag)

        // The container is started with AutoRemove: true, and a restart policy to
        // not restart
        c.Resource, err = pool.RunWithOptions(&dockertest.RunOptions{
            Repository: c.DockerRepo,
            Tag:        c.DockerTag,
            Env:        c.DockerEnvars(),
        }, func(config *docker.HostConfig) {
            config.AutoRemove = true
            config.RestartPolicy = docker.RestartPolicy{
                Name: "no",
            }
        })

        if err != nil {
            log.Fatalf("Could not start container %s:%s: %s", c.DockerRepo,
c.DockerTag, err)
        }

        // Even if everything goes OK, kill off the container after n seconds
        _ = c.Resource.Expire(120)
    }

    // Regardless of whether the DB is docker-based on not, we use the pool's
    // exponential backoff helper to wait until connections succeed for this
    // database
    err = pool.Retry(func() error {
        testConn, err := sql.Open(c.Driver, c.DSN())
        if err != nil {
            return err
        }

        // We close the test connection... other code will re-open via the DSN()
        defer func() { _ = testConn.Close() }()
        return testConn.Ping()
    })
    if err != nil {
        log.Fatalf("Could not connect to %s: %s", c.DSN(), err)
    } else {
        log.Printf("Successfully connected to %s", c.DSN())
    }
}

// Connect creates an additional *database/sql.DB connection for a particular
// test database.
func (c *TestDB) Connect(t *testing.T) *sql.DB {
    db, err := sql.Open(c.Driver, c.DSN())
    if err != nil {
        t.Error(err)
    }
    return db
}

// Cleanup should be called after all tests with a database instance are
// complete. For dockertest-based tests, it deletes the docker containers.
// For SQLite tests, it deletes the database file from the temp directory.
func (c *TestDB) Cleanup(pool *dockertest.Pool) {
    var err error

    switch {
    case c.Driver == SQLiteDriverName:
        err = os.Remove(c.Path())
        if os.IsNotExist(err) {
```

```
            // Ignore error cleaning up nonexistent file
            err = nil
        }

    case c.IsDocker() && c.Resource != nil:
        err = pool.Purge(c.Resource)
    }

    if err != nil {
        log.Fatalf("Could not cleanup %s: %s", c.DSN(), err)
    }
}
}
```

The End

This PDF was generated by gitprint.me